

Architectural Banking System

Course Name: Advanced Programming (OOPS) Lab

Project Title: Mini Architectural Banking System

Name: Shruti Kumari

Roll Number: 25

Instructor Name: Soumadip Biswas

Semester: Spring 2026

2. Problem Statement

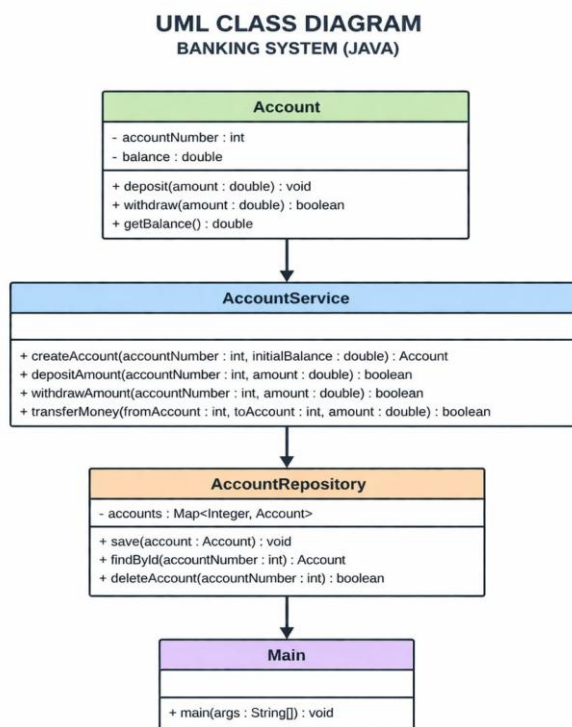
Modern banking systems require efficient and scalable software architectures to manage financial transactions securely. Many traditional systems lack modularity, making them difficult to maintain and extend.

The objective of this project is to design and implement an Architectural Banking System using Object-Oriented Programming (OOP) principles. The system performs core banking operations such as account creation, deposit, withdrawal, and account viewing.

The system is designed using a layered architecture that separates user interaction, business logic, and data management. This ensures better maintainability, scalability, and reusability.

3.System design

3.1 UML class daigram



3.2 Description of Major Classes

1. Account (Model Layer)

Attributes: id, name, balance

Methods: deposit(), withdraw(), getBalance()

Purpose: Represents a bank account

2. Account Repository (Repository Layer)

Stores account data using HashMap

Provides methods to add, retrieve, and delete accounts

3. Account Service (Service Layer)

Handles business logic

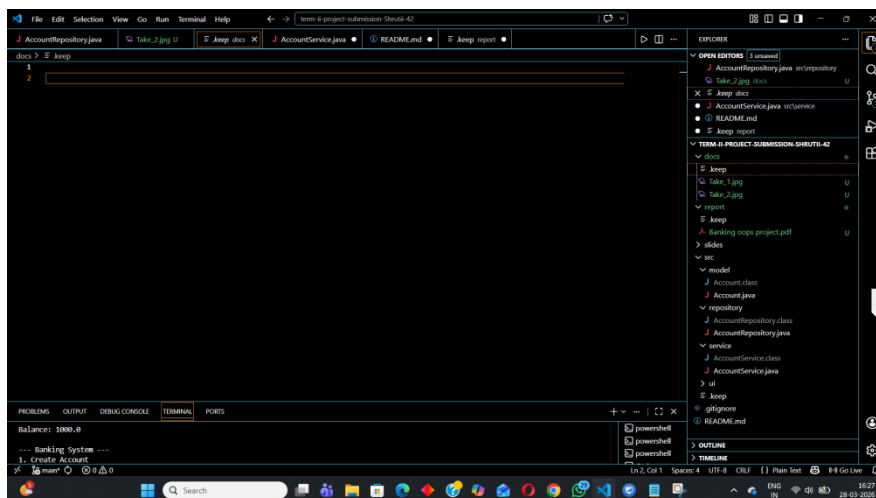
Connects UI with repository

4. Main (UI Layer)

Provides menu-driven interface

Handles user interaction

3.3 Package Structure



Explanation:

The project is divided into four layers:

Model Layer: Defines core data structures

Repository Layer: Handles data storage

Service Layer: Implements business logic

UI Layer: Manages user interaction

This separation improves modularity and scalability.

3.4 Interaction Flow

User selects an operation from the menu

UI sends request to Service layer

Service interacts with Repository

Repository accesses/stores data

Response is returned to user

4. OOP Concepts Demonstrated

4.1 Encapsulation

```
private double balance;  
public double getBalance() {  
    return balance;}  

```

Where it is used: Account class

Why it is used: To protect sensitive data

Explanation:

The balance variable is private and accessed through getter methods, ensuring controlled access and data security.

4.2 Abstraction

```
public void deposit(double amount) {  
    if (amount <= 0) return;  
    balance += amount;  
}
```

Where it is used: Account class

Why it is used: To hide internal implementation

Explanation:

The user does not need to know how deposit works internally, only how to use it.

4.3 Modularity (Layered Design)

`private AccountRepository repo = new AccountRepository();`

Where it is used: AccountService

Why it is used: To separate logic and storage

Explanation:

The service layer interacts with the repository instead of handling data directly.

5. Implementation Highlights

5.1 Deposit Operation

```
public void deposit(double amount) {  
    if (amount <= 0) {  
        System.out.println("Invalid deposit amount");  
        return;  
    }  
    balance += amount;  
}
```

Where it is used: Account class

Why it is used: To validate input before updating balance

Explanation:

Ensures only valid deposits are processed and prevents incorrect data entry.

5.2 Withdrawal Operation

```
public void withdraw(double amount) {  
    if (amount <= 0) return;  
    if (amount > balance) return;  
    balance -= amount;}  

```

Where it is used: Account class

Why it is used: To prevent overdrawing

Explanation:

Validates amount and ensures sufficient balance before withdrawal.

5.3 Repository Storage

```
private Map<Integer, Account> accounts = new HashMap<>();  
  
public void addAccount(Account acc) {  
    accounts.put(acc.getId(), acc);  
}
```

Where it is used: AccountRepository

Why it is used: Efficient data storage

Explanation: Uses HashMap for fast access using account ID.

5.4 Service Layer Logic

```
public void deposit(int id, double amount) {  
    Account acc = repo.getAccount(id);  
    if (acc != null) {  
        acc.deposit(amount);  
    }  
}
```

Where it is used: AccountService

Why it is used: To handle business logic

Explanation:

Ensures operations are performed only on valid accounts.

5.5 User Interface (Menu System)

```
System.out.println("1. Create Account");  
System.out.println("2. View Account");  
System.out.println("3. Deposit");  
System.out.println("4. Withdraw");
```

Where it is used: Main class

Why it is used: To provide user interaction

Explanation:

Displays options and allows users to perform banking operations.

6. Testing & Error Handling

6.1 Test Cases

Create account with valid data

Deposit valid amount

Withdraw within balance

View account

6.2 Edge Cases

Negative deposit

Withdraw exceeding balance

Invalid menu input

6.3 Failure Scenarios

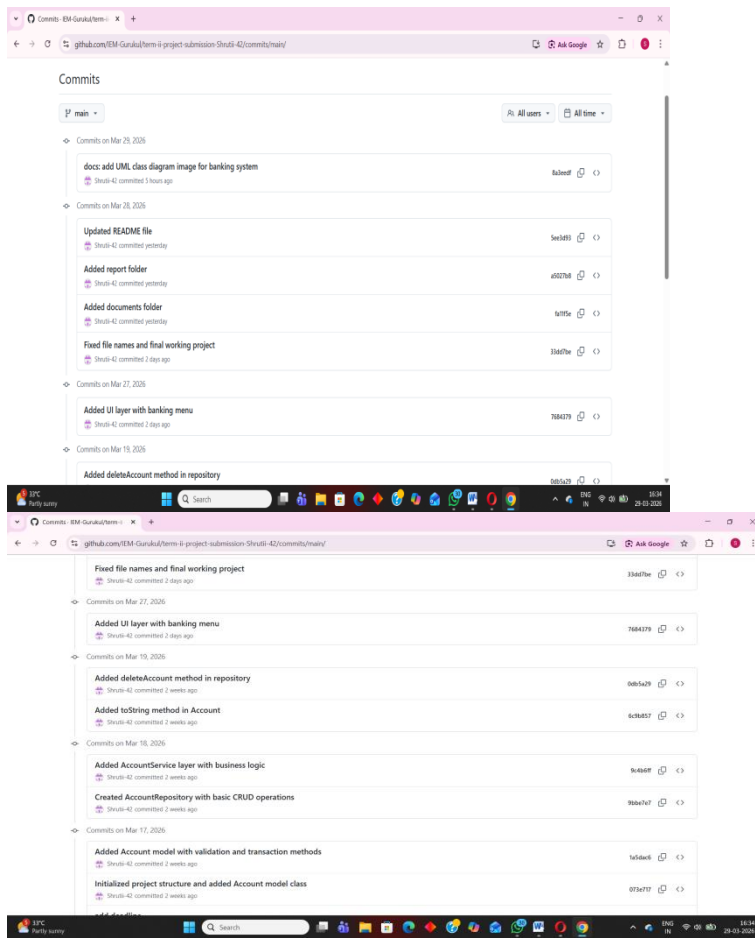
Account not found

Invalid input

System handles errors using conditions

7. Git Workflow

7.1 Commit History



7.2 Development Progression

Created project structure

Implemented model classes

Added repository layer

Implemented service logic

Developed UI

Tested system

8. Conclusion & Future Scope

Conclusion

The Architectural Banking System successfully demonstrates the use of Object-Oriented Programming and layered architecture. The system is modular, maintainable, and efficient in handling basic banking operations.

Future Scope

Database integration

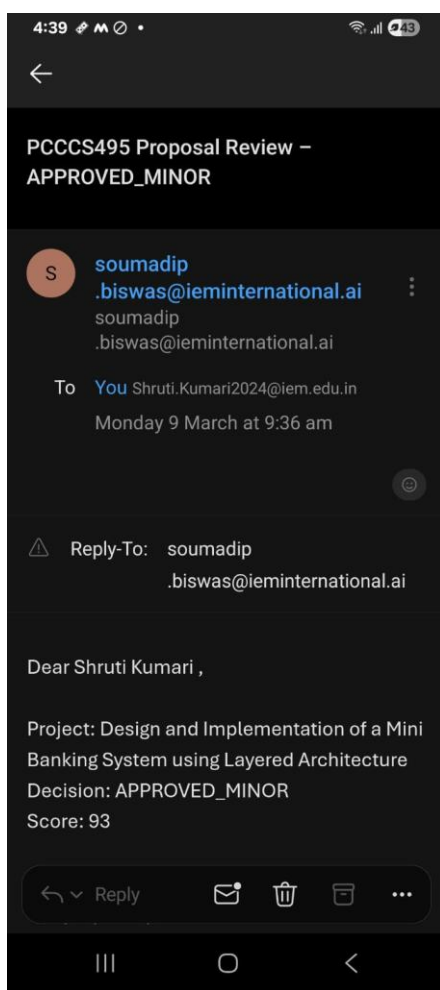
GUI development

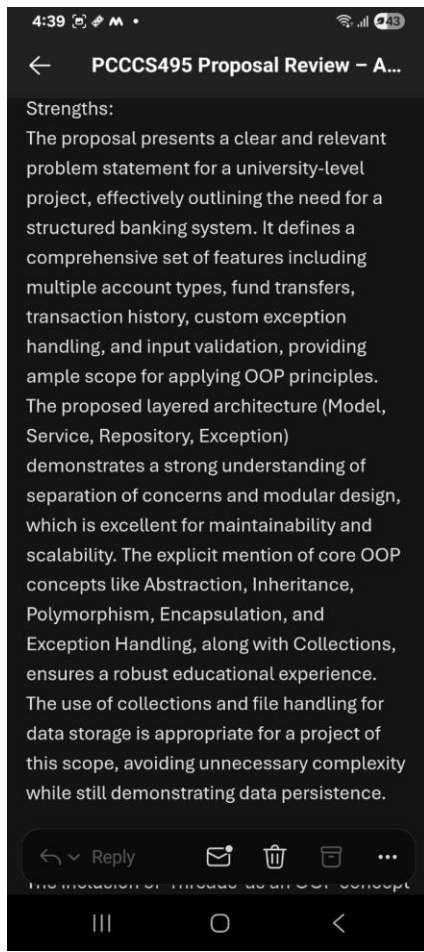
Online banking features

Security improvements

Appendix

A1: proposal approval





Dear Shruti Kumari ,

Project: Design and Implementation of a Mini Banking System using Layered Architecture

Decision: APPROVED_MINOR

Score: 93

Strengths:

The proposal presents a clear and relevant problem statement for a university-level project, effectively outlining the need for a structured banking system. It defines a comprehensive set of features including multiple account types, fund transfers, transaction history, custom exception handling, and input validation, providing ample scope for applying OOP principles. The proposed layered architecture (Model, Service, Repository, Exception) demonstrates a strong understanding of separation of concerns and modular design, which is excellent for maintainability and scalability. The explicit mention of core OOP concepts like Abstraction, Inheritance, Polymorphism, Encapsulation, and Exception Handling, along with Collections, ensures a robust educational experience. The use of collections and file handling for data storage is appropriate for a project of this scope, avoiding unnecessary complexity while still demonstrating data persistence.

Improvements:

The inclusion of 'Threads' as an OOP concept needs clearer justification within the context of a 'Mini Banking System'. It's not immediately evident how multi-threading would be applied to the defined features (e.g., concurrent user access, background processing) without explicitly mentioning such scenarios in the problem or features section. Clarifying the specific use cases for threads would strengthen its relevance or, alternatively, consider removing it if it's not central to the core project objectives to avoid overcomplication. Additionally, while not explicitly required by the prompt, considering a simple user interface (e.g., console-based) and a brief mention of testing strategies would further enhance the project's completeness.

Regards,

Course Instructor